A classe `java.util.Optional` é um objeto contêiner introduzido no Java 8 que serve para representar a **presença ou a ausência de um valor**, ajudando a evitar o temido erro `NullPointerException`. [1, 2, 3]

O Problema que ela Resolve

- **Antigamente (com `null`):** Para saber se um objeto era nulo, o código ficava cheio de verificações manuais (`if (x != null)`), o que poluía a leitura e gerava exceções se você esquecesse de testar. [1, 2, 3, 4]
- **Com `Optional`:** A própria assinatura do método ou tipo deixa explícito que o resultado pode não existir, forçando o programador a tratar esse cenário de forma mais limpa e segura. [1, 2, 3]

Principais Formas de Criar um `Optional`

- `Optional.of(valor)`: Cria um `Optional` com um valor que **não pode ser nulo** (se passar `null`, lança `NullPointerException`).
- `Optional.ofNullable(valor)`: Cria um `Optional` que pode conter um valor ou estar vazio caso o valor seja `null`.
- `Optional.empty()`: Cria um `Optional` vazio explicitamente. [1, 2]

Métodos Mais Utilizados para Consumir o Valor

- `isPresent()`: Retorna `true` se houver um valor e `false` se estiver vazio.
- `isEmpty()`: Retorna `true` se estiver vazio *(disponível a partir do Java 11)*.
- `ifPresent(Consumer)`: Executa uma ação apenas se o valor estiver presente.
- `orElse(outroValor)`: Retorna o valor encapsulado se existir, ou um `outroValor` padrão caso esteja vazio.
- `orElseGet(Supplier)`: Semelhante ao `orElse`, mas executa uma função/supridor apenas se estiver vazio (mais eficiente para processamentos pesados).
- `orElseThrow()`: Retorna o valor ou lança uma exceção (como `NoSuchElementException`) se estiver vazio. [1, 2, 3]

Boas Práticas (Onde usar e onde NÃO usar)

- **Onde usar:**
  - Principalmente como **tipo de retorno de métodos** quando a ausência de um resultado é uma condição normal do negócio (ex: buscar um usuário por ID que pode não existir no banco). [1, 2]
- **Onde NÃO usar:**
  - **Em campos de classes (atributos):** Adiciona peso desnecessário de objetos e não é serializável.

- **Em parâmetros de métodos:** Obriga quem chama o método a embrulhar os argumentos em `Optional.of(...)`, tornando o código ruidoso.
- **Em coleções:** Em vez de retornar `Optional<List<T>>`, retorne uma **lista vazia** (`Collections.emptyList()`). [1, 2]

Você pode conferir mais detalhes e discussões técnicas no Guia para Java Opcional - Baeldung ou ler o artigo original da Oracle Brasil. [1, 2]

Aqui está um exemplo comparando a abordagem tradicional com `null` e a abordagem moderna usando `Optional`.

Cenário: Buscar o nome do país de um usuário

Imagine que temos um objeto `Usuario`, que pode ter um `Endereco`, que por sua vez pode ter um `Pais`. Qualquer uma dessas etapas pode ser nula.

---

1. Abordagem Tradicional (Antes do Java 8)

Para evitar o `NullPointerException`, você precisava fazer várias checagens `if`:

java

```
public String obterNomePaisDoUsuarioTradicional(Usuario usuario) {
    if (usuario != null) {
        Endereco endereco = usuario.getEndereco();
        if (endereco != null) {
            Pais pais = endereco.getPais();
            if (pais != null) {
                String nome = pais.getNome();
                if (nome != null) {
                    return nome.toUpperCase();
                }
            }
        }
    }
    return "PAÍS NÃO ENCONTRADO";
}
```

Use o código com cuidado.

O código sofre do efeito "pirâmide do mal" (vários `if` aninhados) e fica difícil de ler.

---

2. Abordagem Moderna (Com `Optional`)

Se os métodos da sua classe retornarem `Optional`, você pode encadear as operações usando métodos como `map` e resolver tudo em uma única linha fluida:

java

```
public String obterNomePaisDoUsuarioComOptional(Usuario usuario) {
    return Optional.ofNullable(usuario)
            .flatMap(Usuario::getEndereco) // Retorna Optional<Endereco>
            .flatMap(Endereco::getPais)     // Retorna Optional<Pais>
            .map(Pais::getNome)            // Retorna Optional<String>
            .map(String::toUpperCase)      // Transforma o texto se presente
            .orElse("PAÍS NÃO ENCONTRADO"); // Valor padrão caso algo falhe no
caminho
}
```

Use o código com cuidado.

O que está acontecendo aqui?

- **`ofNullable(usuario)`:** Começa o fluxo embrulhando o usuário. Se ele for nulo, o fluxo continua como um `Optional` vazio sem quebrar.
- **`flatMap`:** Usado quando o próprio método (`getEndereco()` ou `getPais()`) já retorna um `Optional`. Ele serve para "desembrulhar" a camada extra e não deixar o código como `Optional<Optional<Endereco>>`.
- **`map`:** Usado quando o método (`getNome()`) retorna um objeto comum (uma `String`). Ele aplica a transformação apenas se o valor estiver presente.
- **`orElse`:** Se qualquer etapa anterior falhar (for vazia ou nula), o Java ignora o restante do pipeline e retorna o texto `"PAÍS NÃO ENCONTRADO"`.